

# PagedAttention 技术调研报告

## PagedAttention 技术调研报告

基于论文“Efficient Memory Management for LLM Serving with PagedAttention” (SOSP 2023)、vLLM 代码库 (83k nodes) 和关联项目的综合分析 生成时间: 2026-06-29

### 1. 论文核心设计

#### 1.1 问题背景

LLM 推理服务的内存瓶颈：

| 显存占用 | 用途       | 特点            |
|------|----------|---------------|
| 65%  | 模型权重     | 静态，不变         |
| 30%  | KV Cache | 动态增长/收缩，随请求变化 |
| 5%   | 激活值      | 临时张量          |

**核心矛盾**：KV Cache 占显存大、动态变化、寿命不可预知，而现有系统要求连续内存存储，导致：

- **内部碎片**：预分配最大长度 (2048 tokens)，实际使用远小于此
- **外部碎片**：不同请求预分配不同大小
- **无共享**：beam search/parallel sampling 的多个序列无法共享 KV cache

**实测**：现有系统仅 **20-38%** 的 KV cache 内存被有效利用。

#### 1.2 核心创新：PagedAttention

受操作系统虚拟内存分页机制启发：

OS 虚拟内存      PagedAttention  
pages      ↔      KV cache blocks

bytes ↔ tokens  
processes ↔ requests

四个关键设计：

1. 分块管理 (**Block-level**)：KV cache 按固定大小 block 分配，不再要求连续内存空间
2. 按需分配 (**On-demand**)：消除内部碎片，不再预分配请求的最大可能长度
3. 等大小块 (**Uniform Size**)：消除外部碎片，所有 block 相同大小
4. 块级共享 (**Block Sharing**)：同一请求的多个序列，甚至不同请求之间，可在 block 粒度共享 KV cache

## 1.3 vLLM 系统架构

基于 PagedAttention 构建的推理引擎：

API Server (FastAPI/OpenAI-compatible)

↓

Scheduler (抢占式调度 + continuous batching)

↓

Block-level Memory Manager (与 PagedAttention 协同设计)

↓

Worker (GPU/NPU kernel execution)

├ PagedAttention V1 (基础版)

└ PagedAttention V2 (多分区版, 类 FlashDecoding)

实现规模：~8K 行 Python + ~2K 行 C++/CUDA

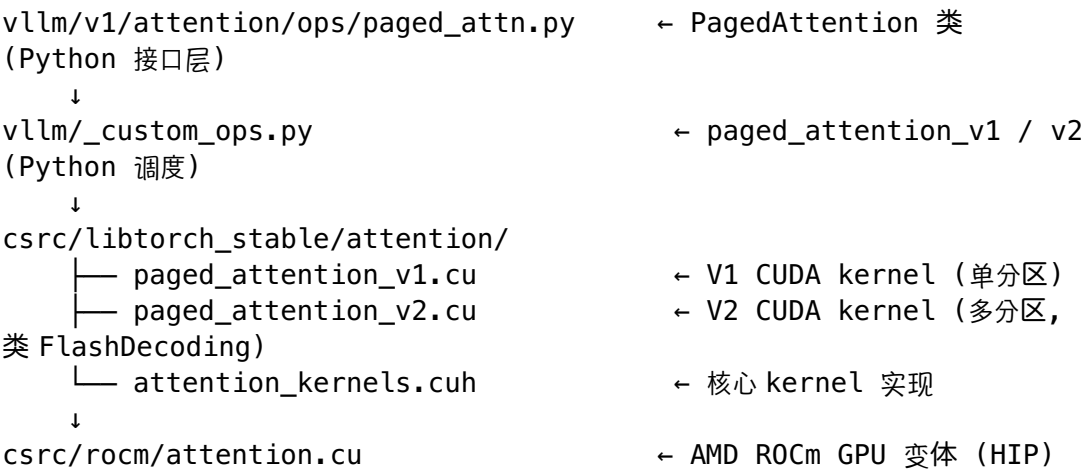
## 1.4 实验结果

| 指标                         | 改进前    | 改进后         |
|----------------------------|--------|-------------|
| KV Cache 利用率               | 20-40% | <b>96%+</b> |
| 吞吐量 (vs FasterTransformer) | 1x     | <b>2-4×</b> |
| 长序列优势                      | —      | 更明显         |
| 复杂解码 (beam search)         | —      | 更明显         |

## 2. 代码实现分析

基于 codebase-memory 对 vllm (83,139 nodes / 490,393 edges) 的深度索引。

2.1 核心实现层级



| 函数 | 文件 | 行数 | 圈复杂度 | 循环深度 | 说明            |
|----|----|----|------|------|---------------|
|    |    |    |      |      | V1 kernel 启动器 |

### 2.3 V1 vs V2 kernel 对比

| 维度      | PagedAttention V1        | PagedAttention V2                         |
|---------|--------------------------|-------------------------------------------|
| Grid 结构 | (num_heads, num_seqs, 1) | (num_heads, num_seqs, max_num_partitions) |
| 分区支持    | 单分区 (no partitioning)    | 多分区 (partition-level parallel)            |
| 灵感来源    | FasterTransformer MHA    | FlashDecoding                             |
| 输出      | 直接写 output tensor        | 先写 tmp_out → reduce kernel 汇总             |
| 适用场景    | 短序列推理                    | 长序列推理 + 高并发                               |

### 2.4 block\_tables 数据结构

所有 kernel 的核心参数：

```
const int* block_tables // [num_seqs, max_num_blocks_per_seq]
// block_tables[i][j] = 第 i 个请求的第 j 个逻辑 block 对应的物理 block 编号
// 类似 OS 页表：逻辑地址 → 物理地址 的映射
```

配合参数：seq\_lens（每个请求的实际长度）、block\_size（每 block 的 token 数）、max\_num\_blocks\_per\_seq（最大 block 数）

### 2.5 性能 benchmark 基础设施

- benchmarks/kernels/benchmark\_paged\_attention.py — PagedAttention kernel 性能测试 (10 个外部依赖)
- benchmarks/benchmark\_prefix\_caching.py — prefix cache 命中率 benchmark
- benchmarks/benchmark\_block\_pool.py — block pool 管理性能

### 3. 关联项目和生态

#### 3.1 直接相关项目



#### 3.2 同类系统对比

| 系统                | 核心技术                       | 论文           | 相对 vLLM          |
|-------------------|----------------------------|--------------|------------------|
| vLLM              | PagedAttention             | SOSP 2023    | —                |
| SGLang            | RadixAttention             | NeurIPS 2024 | 吞吐量 6.4× vs vLLM |
| FasterTransformer | 手写 CUDA kernel             | —            | vLLM 2-4× 超越     |
| Orca              | Iteration-level scheduling | OSDI 2022    | vLLM 显著超越        |

#### 3.3 知识图谱中的论文关联

本 KG 中 vllm-project/pagedattention 论文与其他资源的关联：- 所属标签: vllm, pagedattention, 推理, kv-cache - 关联项目: vllm-project--vllm, vllm-project--vllm-ascend - 跨资源查询: by-tag['推理'] → 2 项目 + 2 论文

## 4. 关键技术要点

### 4.1 内存效率

PagedAttention 的核心价值在于将 **KV cache** 的内存利用率从 **20-40%** 提升到 **96%+**：- 内部碎片消除：block 按需分配，不再预分配最大长度 - 外部碎片消除：所有 block 等大小，buddy allocator 管理 - 共享：prefix caching 通过 block hash 匹配自动复用

### 4.2 Continuous Batching

vLLM 的调度器支持 iteration-level batching，配合 PagedAttention 的灵活内存管理：- 请求到达即加入 batch - 请求完成即释放 block - 不需要等待整个 batch 完成

### 4.3 抢占式调度

当 GPU 显存不足时，vLLM 支持：- 交换 (**Swapping**)：将 block 换出到 CPU 内存 - 重计算 (**Recomputation**)：丢弃 block，需要时重新计算 - 两种策略可根据 latency/throughput 权衡选择

### 4.4 多 GPU 扩展

- Tensor Parallelism: 模型层内分片
- Pipeline Parallelism: 模型层间分片
- PagedAttention 的 block\_tables 在两种并行策略下均有效

## 5. 代码审查要点

基于 codebase-memory 分析，以下是审查 PagedAttention 实现时需要注意的关键模块：

| 审查维度 | 关键文件                                          | 关注点                  |
|------|-----------------------------------------------|----------------------|
| 核心算法 | attention_kernels.cuh::paged_attention_kernel | 411 行，40 复杂度，3 层嵌套循环 |
|      | block_tables 索引逻辑                             | 越界访问检查               |

| 审查维度   | 关键文件                                     | 关注点                                      |
|--------|------------------------------------------|------------------------------------------|
| 内存安全   |                                          |                                          |
| 性能热点   | <code>paged_attention_v2_launcher</code> | Grid 维度计算和 warp 利用率                      |
| 测试覆盖   | <code>tests/kernels/attention/</code>    | <code>ref_paged_attn</code> 出现在 6 个测试文件中 |
| GPU 兼容 | <code>csrc/rocm/attention.cu</code>      | 3 个 kernel 变体, NVIDIA/AMD 双平台            |

## 6. 参考资料

### 论文

- [Efficient Memory Management for Large Language Model Serving with PagedAttention](#) — SOSP 2023
- 中文翻译版: [references/vllm-project/pagedattention/paper\\_cn.pdf](#)

### 代码

- [vllm-project/vllm](#) — 主仓库 (83k nodes)
- [vllm-project/vllm-ascend](#) — Ascend 后端

### 知识图谱内关联

- 论文摘要: [references/vllm-project/pagedattention/summary.md](#)
- 项目摘要: [projects/vllm-project/vllm/summary.md](#)
- 跨资源查询: `by-tag['vllm']` → 6 projects + 1 paper

本报告由 *KG System* 自动生成 | 数据来源: *codebase-memory* + *by-tag* 跨资源  
索引 + 论文翻译